

## Lab 3: Requirements Analysis and Context Modelling

Supporting tools: Visual Paradigm

### Lab objective(s)

Understanding of:

- Reading a narrative system description and separating facts, responsibilities and business rules.
- The **system boundary**: deciding what is inside the system you are building and what is outside it.
- **External entities**: the sources and destinations of data that lie outside the boundary.
- Naming the data that crosses the boundary, in each direction.
- Constructing and validating a **context diagram**.
- Why internal processes and stored data never appear on a context diagram.

Course learning outcome: CLO 2 (elicit, analyse, prioritise and document requirements for a software system).

### Where this fits in software engineering

Before anything can be designed, one question has to be answered: **what are we building, and what is somebody else's problem?**

That question is not a formality. Every later model inherits the answer. If you decide that a neighboring system is part of yours, you will spend the rest of the semester modelling classes you do not have to write, drawing messages to objects you do not control, and testing behaviors that is not yours. No later lab repairs a boundary error, it propagates.

This lab produces one artifact, the **context diagram**, and that artifact becomes a *contract*. In Lab 4 you will open the system up and model what happens inside it, and you will not be allowed to silently invent, remove, rename or reverse anything crossing the boundary you draw today.

### A note on levels

Data flow diagrams are drawn in levels, and different books number them differently. **This course uses the following convention throughout**, so that the level number always matches the depth of the process numbers:

| Diagram               | Level          | Contains                                                       | Process numbers |
|-----------------------|----------------|----------------------------------------------------------------|-----------------|
| Context diagram       | <b>Level 0</b> | The whole system as <b>one</b> process, plus external entities | 0               |
| First decomposition   | <b>Level 1</b> | The system's major processes                                   | 1, 2, 3...      |
| Further decomposition | <b>Level 2</b> | The sub-processes of one Level 1 process                       | 5.1, 5.2...     |

This lab produces the Level 0 diagram. Lab 4 produces Level 1 and one Level 2.

## Part A: QU Print Shop

Before moving on to the more extensive example in Part B, complete this short exercise to develop a clear understanding of the lab process. Working with the instructor, annotate the system description to identify external entities, data, functions, business rules and out-of-scope responsibilities. Then determine the system boundary, list the data flows entering and leaving the system, and use them to construct and validate a Level 0 context diagram. Discuss and check each answer as you progress.

### A.1 The system

Fill each empty box with the appropriate symbol: **E** (external entity), **D** (data), **F** (function), **R** (business rule), or **X** (outside scope).

The QU Print Shop [ ] lets students submit documents for printing. A [ ] student uploads [ ] a file and [ ] chooses printing options: the number of copies, colour or black and white, and single- or double-sided printing.

Before accepting a job, the Print Shop [ ] checks whether the student has sufficient credit. [ ] Credit is held by the [ ] **Print Account System (PAS)**, a commercial package already operated by the university. [ ] The Print Shop does not hold balances or take money. After printing a job, the Print Shop [ ] sends [ ] charge advice to PAS, and [ ] PAS processes the charge.

The Print Shop [ ] keeps a record of every print job, including [ ] who submitted it, the selected printing options, and whether its status is queued, printed, or collected. When a job is ready, the Print Shop [ ] notifies the [ ] student, who submits [ ] a collection request when collecting it.

### A.2 Deciding the boundary

Three questions settle most boundary decisions:

1. **Do we build it?** If it already exists and we are not writing it, it is outside.
2. **Do we own its data?** If another system is the authoritative record, it is outside.
3. **Does it send us data, or receive data from us?** If yes, and it is outside, it is an **external entity**.

Apply them to PAS. The university already runs it, it owns the balances, and the Print Shop exchanges data with it. **PAS is outside the boundary and is an external entity.**

Notice what the third question does *not* ask. It does not ask whether the two systems belong to the same organization. Common ownership is irrelevant; the boundary is a decision about what you are building.

| Element                           | Inside or outside? | Because                                            |
|-----------------------------------|--------------------|----------------------------------------------------|
| Recording and tracking print jobs | Inside             | The Print Shop is the authoritative record of jobs |
| Producing the printed output      | Inside             | It is the reason the system exists                 |
| Holding student credit balances   | <b>Outside</b>     | PAS owns this data                                 |
| Processing the actual charge      | <b>Outside</b>     | PAS performs it; we only advise                    |
| The student                       | <b>Outside</b>     | A person, interacting with the system              |

### A.3 External entities

Only two: **Student** and **PAS**.

Things that are *not* external entities, and why:

| Candidate  | What it really is                                               |
|------------|-----------------------------------------------------------------|
| Print job  | <b>Data</b> the system stores a data store, in Lab 4            |
| Job number | An <b>attribute</b> of a print job                              |
| Printer    | An implementation detail. A logical model does not name devices |

| Candidate   | What it really is                         |
|-------------|-------------------------------------------|
| Print queue | <b>Stored data</b> , not an outside party |

**Rule.** An external entity must be able to *originate* data or *receive* data, from outside the system, of its own accord. A thing the system merely remembers is data, not an entity.

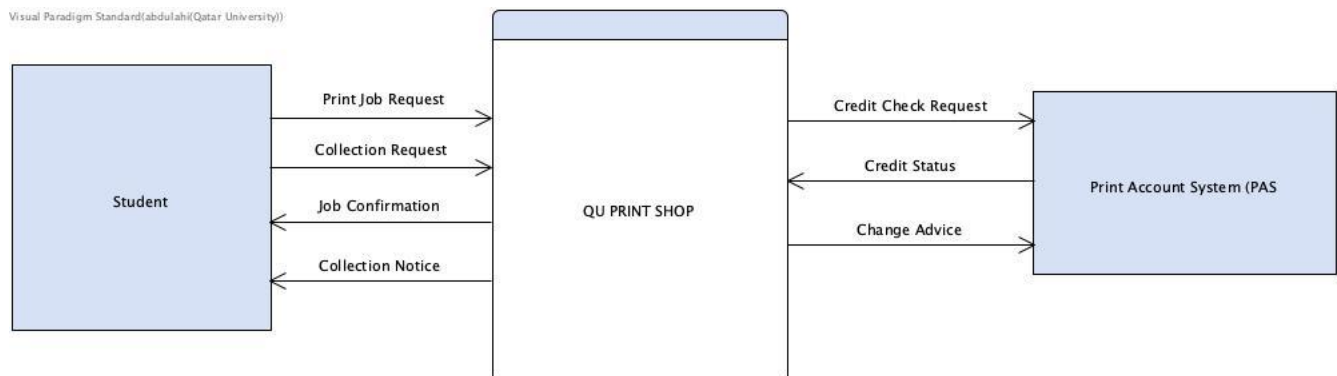
#### A.4 The data that crosses the boundary

Every arrow on a data flow diagram carries **data**, so every arrow is named with a **noun phrase**. *Print Job Request*: yes. *Submit Job*: no; that is an action, and actions are what processes do, not what arrows carry.

Use one arrow per direction. A single two-headed arrow labelled vaguely hides the fact that different data moves each way.

| Source     | Data flow            | Destination |
|------------|----------------------|-------------|
| Student    | Print Job Request    | Print Shop  |
| Print Shop | Credit Check Request | PAS         |
| PAS        | Credit Status        | Print Shop  |
| Print Shop | Job Confirmation     | Student     |
| Print Shop | Collection Notice    | Student     |
| Student    | Collection Request   | Print Shop  |
| Print Shop | Charge Advice        | PAS         |

#### A.5 The context diagram



Three properties define a context diagram, and all three are checkable at a glance:

- **Exactly one process**, named after the system.
- **No data stores**. Storage is internal; at Level 0 the system is opaque.
- **Every arrow crosses the boundary**. Nothing internal appears, because there is no "internal" yet.

**Checkpoint.** Why are there no data stores on a context diagram, even though the Print Shop clearly stores print jobs? *Because a data store is inside the system, and at Level 0 the system is a single closed box. Showing a store would mean showing part of the inside.*

## Part B: The All Phones Shop

---

Part B is your assessed work. Work in groups of three for B.1 to B.3; **B.4 is individual**. Read the description of the system carefully.

### The system description

---

All Phones Shop (APS) is a company that sells three types of phones: *smartphone*, *normal mobile phone*, and *land phone*. APS currently uses a small computer-based system called PoB that manages the payroll of its employees, all payments made by clients, and the financial information of the APS clients. PoB is actually a standard, commercial-off-the-shelf application. APS has two types of clients: *individual*, and *corporate*.

To buy or sell a phone, the client must first register if he/she does not have any registration with APS. In order to register, the client has to provide name and address. APS first checks if the client already exists. If not, the system creates a registration, assigns a unique registration number. PoB is advised by APS with the registration number. PoB then opens a financial account of the client with the registration number. All payments of the client are handled by PoB and recorded in the financial account. However, APS keeps only the registration information such as name, address, the registration number, the invoice(s) if the client has bought any item from APS, and the item information if the client sold any item. For all future sell and buy activities with APS, the client has to use the same registration number.

A registration can only be removed by corporate clients. In this case, APS checks if there is any unpaid invoice attached with the client registration. PoB is informed if a client registration is removed. The individual client can only create registration but cannot remove any registration. APS does not handle any login operation; it is assumed all login functions are checked before any operation is executed.

To sell an item, the client first selects the type of the item that he/she wants to sell. Under smartphone category, there are two sub-categories: *Apple iPhone* and *Samsung Galaxy*. The client then provides model, make, capacity, and color. The client has the option to sell the item either on auction (bidding), or on fixed price sale. For the auction, a starting price and the end of the auction time is specified; and for the fixed price sale, a price is given. For both types of sale (fixed price and auction), the system assigns an ID to the item, records and attaches the item information with the client registration.

For the catalog entry, APS then automatically creates a display image based on the item description, asks the client (seller) to approve it, and displays the item for sale if approved by the client. Otherwise, the system re-creates another image, and displays it to the client for approval. Once the client approves the display image, APS includes it in its product catalog.

Clients can also remove their items after display but before sale; in that case, a penalty of 10% of the starting price (for auction) or the fixed price of the item is charged to the client, and an advice on this penalty will be sent to PoB.

Clients can also buy items on auction, or on fixed price sale. For the fixed price items, the client selects item(s). The system creates a shopping basket for the selected item(s). The system saves the shopping basket and attaches this with the client registration. The client then checks out, and each item in the basket is considered "sold" and removed from the product catalog.

For auction, clients can bid a price for the selected item before the end of auction time. A bid submitted by a client must be greater than the last offered bid (price). All bids submitted by a client are recorded in the registration of the client. When the auction time ends, the client with the last bid will be notified as the winner. The item is considered "sold" and removed from the product catalog.

For either type of sale, APS prepares an invoice (bill) based on the item(s). Once the client confirms the invoice, it is attached with the client registration with the status "unpaid".

For all payments, PoB informs APS with the client registration number and the invoice number. The system finds the invoice and assigns the invoice as "paid". If a client fails to pay to PoB within 7 days, a penalty of 10% of the price is charged to the client, an advice on this penalty is sent to PoB, and the sale is cancelled by APS.

### B.1 Annotate the description (25 min)

Read the case individually and mark it up:

| Mark | Meaning                                     |
|------|---------------------------------------------|
| E    | a candidate external entity                 |
| D    | data the system must remember               |
| F    | something the system must do                |
| R    | a business rule or constraint               |
| X    | a sentence that puts something out of scope |

There are at least two sentences that explicitly place something out of scope. Find both and quote them you will need them in B.2.

### B.2 Decide the boundary (25 min)

In your group, complete this table. **Every row needs a justification**, and where the description settles it, quote the sentence.

| Element                          | Inside / outside | Because...                             |
|----------------------------------|------------------|----------------------------------------|
| The client                       | Outside          | A person, interacting with the system. |
| Holding Registration Information | Inside           |                                        |
| Client Information               | Inside           |                                        |
| Auction and Bidding              | Inside           |                                        |
| PoB System                       | Outside          |                                        |
| Shopping Basket                  | Inside           |                                        |
| Financial Account                | Outside          |                                        |
| Login Operations                 | Outside          |                                        |

Then write a **boundary statement** of two to four sentences: what APS is responsible for, and what it deliberately leaves to others.

**Warning.** Two of these rows are settled explicitly by the text. If your group is *debating* them, you have not found the sentence yet.

### B.3 External entities and boundary data (25 min)

**Step 1 entities.** List the external entities. For each rejected candidate, say what it is instead (data, a stored record, an attribute, an implementation detail).

**Step 2 the flow inventory.** Complete this table. Noun phrases only; one row per direction.

| Source | Data flow | Destination | Evidence from the case |
|--------|-----------|-------------|------------------------|
|        |           |             |                        |

Aim for coverage of all six areas of the case: registration, listing an item, the catalogue, buying at a fixed price, buying at auction, and invoicing and payment. If an area has no arrow, you have missed something.

## B.4 Draw the context diagram individually (25 min)

---

In Visual Paradigm: [Diagram > New > Data Flow Diagram](#). Name it [APS Context \(Level 0\)](#).

1. One process, named [All Phones Shop](#).
2. Your external entities, outside it.
3. One named arrow per flow from your inventory.
4. **No data stores. No internal processes.**

You may consolidate closely related flows to keep the diagram readable - [Item and Sale Details](#) may reasonably carry the item description and the chosen sale terms together. You may **not** consolidate into vague labels such as [Information](#), [Data](#) or [Details](#).

Export as [aps-context.png](#) via the Action Bar.

## B.5 Peer check (10min)

---

Swap with a neighbour and apply four tests. Write down one strength and one required correction.

| Test               | Question                                                        |
|--------------------|-----------------------------------------------------------------|
| <b>Boundary</b>    | Does every arrow cross the boundary?                            |
| <b>Source</b>      | Does every incoming arrow come from a legitimate outside party? |
| <b>Destination</b> | Does every outgoing arrow go to one?                            |
| <b>Coverage</b>    | Are all six areas of the case represented?                      |

## B.6 Consolidation and submission (25 min)

---

Submit the document that contains the solution, to your individual repository under [your-repo/03-context/](#):

The document must contain the following four sections:

1. **Boundary analysis**
  - Completed B.2 boundary table
  - Boundary statement
2. **External entities and data flows**
  - Completed B.3 external-entity analysis
  - Completed flow-inventory table
3. **Context diagram**
  - The APS Level 0 context diagram using Gane–Sarson notation
4. **Peer review**
  - One strength identified by your peer
  - One required correction
  - One detail deliberately omitted from the diagram, with an explanation

Together these four sections are your **APS External Contract**. Bring it to Lab 4.

## Assessment : 10 points

---

| Criterion                                                                         | Points |
|-----------------------------------------------------------------------------------|--------|
| Boundary decisions correct and justified, with the two explicit exclusions quoted | 2.5    |
| External entities correct; rejected candidates correctly reclassified             | 1.5    |
| Flow inventory covers all six areas, with correct directions                      | 2      |
| Context diagram has exactly one process, no stores, no internal processes         | 2      |
| All flows named with meaningful noun phrases                                      | 1      |
| Peer correction incorporated; omission question answered                          | 1      |

**Readiness for Lab 4:** you must have a context diagram containing the correct external entities and a defensible set of named boundary flows. If yours does not, correct it before the next session.

### Further reading

- [Visual Paradigm- What is a system context diagram?](#)
- [Visual Paradigm - Data flow diagram levels](#)
- Pressman, *Software Engineering: A Practitioner's Approach* flow-oriented modelling.